

Csound on Embedded Risc V and Xtensa Architectures

Aman Jagwani¹ and Victor Lazzarini¹ *

Maynooth University
aman.jagwani.2023@mumail.ie

Abstract. With Csound 7, support for bare-metal microcontrollers with ARM Cortex CPUs was introduced. This allowed Csound applications to target these embedded platforms without OS dependencies for the first time, expanding the possibilities in areas such as custom instrument design, interactive art installations, and DIY performance systems. This paper extends this work to two more embedded architectures, RISC-V and Xtensa, through the introduction of Csound for the ESP32 series of microcontrollers. These chips are widely available, very inexpensive, low-power, usually dual-core, and support extensive wireless networking capabilities. They also run FreeRTOS, providing an unexplored middle ground between bare-metal systems without schedulers and complete operating systems. This balance between extensive features and accessibility makes them an ideal target for expanding Csound’s range of supported platforms. In this paper, we describe the process of porting Csound to these platforms, highlight their benefits and limitations in relation to previously supported platforms, and discuss potential use cases and applications of Csound on these platforms.

Keywords: Embedded Systems, ESP32, Risc V, Xtensa, FreeRTOS

1 Introduction

With Csound 7, we worked on the support for bare-metal ARM-based platforms [1]. To do this, we first removed OS dependencies such as `libsndfile`, threading and networking. Then we cross-compiled Csound with the ARM embedded toolchain and carefully managed memory, both for storing the Csound application program and for run-time heap allocations. Finally, we provided connections between Csound and the embedded audio hardware through the Csound API. As a result, Csound is now able to target small, portable, inexpensive, audio-application-oriented chips. We developed implementations for three bare-metal platforms, each of which highlights a different point, in terms of processing power and connectivity, on the range of Csound’s current bare-metal capability. Across these platforms, we identified gaps that led to this work: the addition of Csound support for ESP32 chips.

The first bare-metal platform for Csound was the Electrosmith Daisy [1]. This board contains an ARM Cortex-M7 CPU with an on-board audio codec, enabling audio input and output without requiring external hardware connections. This made it a good first step for experimenting with bare-metal Csound, since audio was taken care of and we could focus on the platform-specific Csound implementation. Additionally, the complete Csound library requires almost 2 MB of flash storage for the application binary and more than 1 MB of RAM for its heap [2]. Both of these were available in the QSPI flash and SDRAM of the Daisy, respectively. However, the Daisy is audio-specific and can connect to only a limited set of external peripherals. It has no networking or display capabilities, for example. It has a low barrier to entry but can be restricted in terms of connectivity.

The second bare-metal platform was the Zynq-7000 FPGA SoC (Field-Programmable Gate Array System on Chip) [1]. This chip includes an ARM Cortex-A9 processing system (PS) coupled with programmable logic (PL), or FPGA hardware. Csound runs on the ARM PS, and accelerated versions of Csound opcodes such as `reverb` and the `pvs` (phase vocoder streaming) opcodes run on the PL in tandem, allowing the system to leverage the higher sample rates, parallelism, and low latency of FPGA hardware. While this platform is extremely powerful, it is still complex and requires specialised low-level programming to leverage the FPGA’s capabilities. Additionally, FPGA SoCs can be relatively expensive. It therefore sits at the high-performance but high-complexity and high-cost end of the supported bare-metal platforms.

* Aman Jagwani would like to acknowledge the support of the Hume Scholarship Programme, Maynooth University.

The third bare-metal platform was a generic prototyping board, the STM32H747I Discovery kit [2]. These types of boards are general-purpose and contain a complete set of peripherals, not just audio, such as Ethernet, display, and USB, allowing the prototyping of audio-peripheral connections and coexistence. Through this project, we were able to re-implement Csound’s UDP server as a single-threaded embedded implementation and run it in conjunction with a MIPI DSI on-board display, demonstrating a standalone networked and visual embedded music system centered around Csound. We also utilised the two cores of the board to separate tasks and optimize their usage. The slower Cortex-M4 managed MIDI I/O, networking, and display control, while the faster Cortex-M7 focused solely on audio. This board is the most connected of the three, but also entails substantial manual, low-level implementation effort especially in implementing configuration and transfer of audio from Csound to the audio codec..

The limitations highlighted across the above three platforms present a gap that can be filled by the new platform that we explored and developed in this work: the ESP32 running FreeRTOS on the RISC-V and Xtensa architectures. The ESP32 [3] is a ubiquitous, low-cost, low-power microcontroller platform by Espressif. Its chips are 32-bit and, across the different variants, are based on either the Xtensa [4] or RISC-V [5] architecture. They contain a rich set of peripherals and communication protocols and are especially well-suited to wireless networking, with comprehensive WiFi and Bluetooth support, making them popular choices for Internet of Things (IoT) devices. Some of the ESP32 variants, such as the base model, may not be fast enough or have enough memory to support Csound. However, newer variants such as the ESP32-S3 and ESP32-P4, which we used in this project, are able to support it.

Additionally, ESP32 applications are generally FreeRTOS-based. FreeRTOS is a lightweight real-time operating system (RTOS) for microcontroller platforms [6]. It has a small memory footprint, provides fast execution, and facilitates features such as symmetric multi-processing (SMP), making it simple to work with dual-core architectures like most ESP32s and to schedule tasks across the cores. This, combined with the ESP32-provided connectivity and drivers for audio, USB, UART (Universal Asynchronous Receiver-Transmitter), display interfaces and especially WiFi networking, which are relatively easy to use compared to the STM32, for example, makes it an ideal middle ground between bare-metal and OS-based platforms, offering extensive built-in connectivity and OS-level services while remaining inexpensive, lightweight and accessible. As a Csound target, this opens up opportunities in application areas such as networked wireless audio interaction and smart-home audio devices such as smart speakers.

2 The ESP32 Platform

Various audio projects have utilised the ESP32 as their microcontroller platform. CTAG-TBD [8] is an open source Eurorack module platform based on this chip. It contains an audio plugin architecture that allows users to create their own DSP modules. It also supports a web UI. The inexpensive ESP32 along with the open source hardware and software make this an accessible platform for users to experiment with custom audio processing and sound synthesis in a modular synthesizer context.

Another previous project based around this chip is the Interactive Audio Toolkit [9]. This toolkit connects audience interactions via the ESP32’s peripheral connections over I2C, UART, I2S and ADC interfaces with musical interaction and compositional processes through a sequencer-based interface. It also demonstrates how interactive systems can be designed and controlled through a web-server hosted on the ESP32 for convenient on-site configuration.

Both these projects highlight the benefits of the ESP32’s low-cost and connectivity in embedded audio environments. However, both are limited in their DSP capabilities by the audio modules or plugins implemented from scratch for these respective platforms. With Csound on the ESP32, and its vast, mature DSP library as well as its strong legacy of compositions, installations, and instruments, this limitation can be overcome, and the benefits of the ESP32 can be further leveraged.

The ESP32 contains several variants that are based on either Xtensa or Risc V architectures. Due to the availability of sufficient memory and processing power, we chose the ESP32-S3 and ESP32-P4 variants for experimentation and development.

Table 1. Per-block CPU utilisation of a generative FM CSD on both ESP32 variants.

ESP32 Variant	Per-block Render Time (μ s)	Per-block CPU Utilisation
ESP32-P4	≈ 423	$\approx 16.0\%$
ESP32-S3	≈ 1502	$\approx 56.3\%$

2.1 ESP32P4

The P4 is one of the most powerful ESP32 variants [10]. We used the wave-share module-dev-kit [11] during development. At the time of writing it cost around \$20, while simpler development boards and the microcontroller chip itself cost significantly less. The P4 contains two Risc V cores, that run at up to 400Mhz, and a shared single-precision floating point unit. It contains limited on chip SRAM (768 KB) but sufficient on-package PSRAM (32 MB on our development board) to allow for Csound’s heap. It also contains 16MB flash storage, more than enough for Csound application binaries.

Additionally, the ESP32-P4 is usually paired with a dedicated ESP32-C6 co-processor chip for wireless networking through WIFI and bluetooth. This offload of functionality leaves more processing power on the main P4 chip for audio processing. The C6 communicates with the P4 over a SDIO interface.

2.2 ESP32S3

The S3 variant[12] is less powerful than the P4 and is more catered to general-purpose applications as opposed to high-performance applications in the case of the P4. However, it is significantly cheaper than the P4 with the ESP32S3-N16R8-WROOM-32-1 development board that we used for experimentation, costing around \$8 at the time of writing. Additionally, it has integrated WIFI and bluetooth capabilities and does not require a co-processor for these functionalities like the P4 does. This chip contains two Xtensa LX7 cores that can run at up to 240 MHz, once again with a shared single-precision floating point unit and enough memory, 16MB on-board PSRAM and 8MB flash, for Csound.

2.3 Memory Access and CPU Performance

Both chips contain the required baseline processing power and memory to run Csound and provide two different entry points in the price vs performance tradeoff. However, one key distinction is in the speed of memory access across the two variants. The P4’s PSRAM is accessed over a 16-line interface clocked at 200MHz, while the S3’s PSRAM is accessed over an 8-line interface at 80MHz. This coupled with the difference in processor speed, reduces the capacity of the S3 significantly as compared to the P4.

Table 1 highlights the per-block CPU utilisation of the two variants while running an instance of the same CSD containing a generative FM instrument and a `reverb2` effect instrument. It can be seen that the processing capacity of the P4 is significantly higher than that of the S3. The per-block CPU utilisation was measured by calculating the time taken for the Csound instance to execute `csoundPerformKsmps()` and dividing this time by the total time budget to compute one block of audio. The buffer size for both variants was set to be equal to `ksmps` at 128 and the sample-rate was set at 48000Hz.

Furthermore, Table 2 presents the maximum number of voices that can be run without under-runs or glitches on a single core of each variant, measured with two synthesis techniques: subtractive and FM synthesis. In the case of subtractive synthesis, each voice contained two detuned `vco2` oscillators going through a `moogladder2` filter and in the case of FM synthesis, each voice contained a single carrier-modulator pair. Once again, it can be seen that the P4 with its higher processing power and faster memory access is able to accommodate a substantially higher number of voices. The audio buffer size was 128 and the sample-rate was 48000Hz in this example as well.

Overall, with both variants the trade off between computational load and buffer size needs to be managed carefully. With simpler DSP operations in a CSD, buffer size and latency can be reduced on both chips. However, for more complex DSP operations, a larger buffer size may be required.

Table 2. Maximum subtractive and FM synthesis voice count across the two variants.

ESP32 Variant	Synthesis Technique	Max Voices
ESP32-P4	Subtractive	11
	FM	28
ESP32-S3	Subtractive	5
	FM	9

Table 3. Per-block CPU utilisation of a single `reverb_sc2` instance across single and double-precision builds with buffer size = 128 and sample-rate = 48000Hz. Values above 100 % cannot be rendered in real time.

ESP32 Variant	Single-precision Build	Double-precision Build
ESP32-P4	$\approx 9.5\%$	$\approx 139.3\%$
ESP32-S3	$\approx 18.6\%$	$\approx 145.7\%$

2.4 Cross-compilation

For both ESP32 variants the cross compilation process was similar to the bare-metal build, with minor changes to target and architecture specific flags in the toolchain file. Csound is compiled as a static library with the RISC v and Xtensa toolchains and is then loaded in the Espressif IOT Development framework (IDF) [13] as a component and finally linked to by the application program.

The ease of transfer to these platforms highlights the portability of Csound following the bare-metal development.

However, one major change that this platform required was the use of solely single-precision floating-point operations. The hardware FPU on the ESP32 chips cannot handle double-precision operations, causing significant reductions in performance and speed. For instance, the `reverb_sc2` opcode exceeds the per-block render time and CPU utilisation budgets on both cores when compiled with double-precision internal operations. However, with all of its internal operations and variables converted from double to float, both variants are able to run it with substantial processing headroom. This is highlighted in Table 3.

It is important to note that the conversion from double to single-precision floating point operations results in a slight loss of precision and can decrease the signal-to-noise ratio of filters. However, to be able to run comprehensive Csound programs on an inexpensive, connectivity-rich, and ubiquitous but limited platform like the ESP32, this trade-off may be justified.

3 Real-time Audio With FreeRTOS

FreeRTOS provides tasks, which are like self-contained threads or programs with their own execution context, stack, and priority level [6]. For our Csound implementation, audio processing runs in its own high-priority task. The scheduler then gives it precedence over lower-priority tasks.

Within the audio task, we use the Espressif-provided I2S driver [14] to manage the audio transfers between an audio codec and the ESP32. Similar to the Daisy and STM32H747I-DISCO, these transfers are timed by the DMA interrupts raised as the audio buffers are filled and drained, in sync with the codec’s consumption of the I2S sample stream at the sample rate. However, the distinction lies in the fact that for the Daisy and STM32, the audio processing happens within the interrupt service routine (ISR) context, while here it happens within the context of a task. Nevertheless, the timing is still tightly coupled with the audio hardware due to the high priority of the audio task and the fact that the I2S driver owns the DMA and interrupts internally, and its audio transfer functions are controlled by that same sample-rate clock.

This deferred configuration is important because on the ESP32 platforms, ISRs need to be short, non-blocking, and should not contain floating-point operations, as the FPU state belongs to the interrupted task. So, they contain notifications that defer the processing to a task rather than performing it themselves. By keeping the ISR short, and free of flash-resident code, both watchdog time-outs and the cache stalls are prevented. [15].

Other functionalities, such as MIDI data reading and writing, sensor data polling and processing, and wireless networking, are also executed in their own dedicated tasks, but they are assigned lower priority levels than audio. We implemented lock-free circular buffers for communicating data between tasks.

4 Dual Core

Both the P4 and the S3 variants of the ESP32 contain two cores, and with FreeRTOS, any task can be assigned to either core within the same application program. The application context, scheduler, memory, and floating-point unit are therefore shared across both cores. This enables tasks to be divided efficiently and easily across the two cores with relative ease.

As a general design, for both variants, we reserved one of the cores (core 1) for Csound’s audio processing and placed MIDI, sensor, and network-related tasks on the second core (core 0). However, with sufficient processing headroom, a second instance of Csound can be instantiated on core 0 as well. This can increase the processing power dedicated to Csound, resulting in additional voices for polyphonic synthesizers or additional effects instances, for example. This design results in two separate Csound FreeRTOS tasks, both running at the highest priority, one on each core. However, only the primary instance (core 1) deals with passing audio in and out of the audio codec. The secondary instance passes its resultant audio to the primary instance’s task through a dedicated lock-free audio circular buffer. Then after a mixing stage, the combined audio output is transferred via the I2S driver to the audio codec.

Figure 1 highlights the system architecture, peripheral connections and task division in the context of the P4. For the S3, the architecture is largely the same, other than the networking functionalities being placed on Core 0 instead of the C6 co-processor.

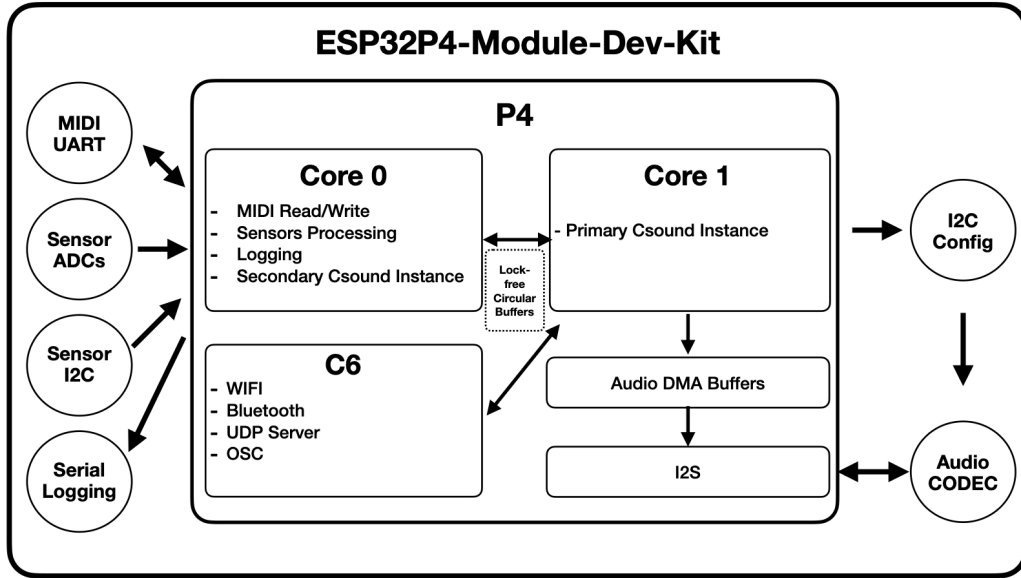


Fig. 1. ESP32P4 Csound System Architecture.

5 Conclusions

This work presented developments for Csound across two new architectures, Risc V and Xtensa as well as strategies and design considerations for Csound within the FreeRTOS ecosystem. The inexpensive cost, vast connectivity options and particularly the wireless communication possibilities of the ESP32 chips, in combination with Csound, can open up opportunities for several musical application areas such as networked interactive installations, web interfaces for embedded instruments and smart home audio devices. The cross-compilation files and example projects for both architectures will be available in the main Csound github repository.

References

1. Lazzarini, V., Jagwani, A.: Csound 7: Bare Metal and Co-Processing Hardwares. In: Proceedings of the 19th Linux Audio Conference (LAC-25). Lyon, France (2025). <https://hal.science/hal-05096036>
2. Jagwani, A., Lazzarini, V.: Bare-metal Csound on Generic Prototyping Platforms. In: Proceedings of the 20th Linux Audio Conference (LAC-26). Maynooth, Ireland (2026)
3. Espressif: ESP32 SoC. <https://www.espressif.com/en/products/socs/esp32>. Accessed June 12, 2026
4. Cadence Tensilica: Xtensa Instruction Set Architecture (ISA) Reference Manual. https://dl.espressif.com/github_assets/espressif/xtensa-isa-doc/releases/download/latest/Xtensa.pdf. Accessed 2026
5. RISC-V International. <https://riscv.org/>. Accessed June 16, 2026
6. FreeRTOS Real-Time Operating System. <https://www.freertos.org/>. Accessed June 12, 2026
7. Keller, D., Schiavoni, F., Lazzarini, V.: Ubiquitous music: Perspectives and challenges. *Journal of New Music Research* 48(2), 309–315 (2019)
8. CTAG-TBD: Trigger-Boum-Digital, an open-source ESP32-based Eurorack audio/synthesis platform. <https://github.com/ctag-fh-kiel/ctag-tbd>. Accessed June 30, 2026
9. Jagwani, A., Lazzarini, V.: Interactive audio toolkit: Creating sonic experiences and installations with low-cost, low-power microcontrollers. *Arts & Communication* 4(2), 025180032 (2026). <https://doi.org/10.36922/AC025180032>
10. Espressif: ESP32-P4 SoC. <https://www.espressif.com/en/products/socs/esp32-p4>. Accessed June 30, 2026
11. Waveshare: ESP32-P4-Module-DEV-KIT Documentation. <https://docs.waveshare.com/ESP32-P4-Module-DEV-KIT>. Accessed June 30, 2026
12. Espressif: ESP32-S3 SoC. <https://www.espressif.com/en/products/socs/esp32-s3>. Accessed June 30, 2026
13. Espressif: ESP-IDF Release 6.0 Migration Guide — System. ESP-IDF Programming Guide. <https://docs.espressif.com/projects/esp-idf/en/stable/esp32/migration-guides/release-6.x/6.0/system.html>. Accessed June 30, 2026
14. Espressif: Inter-IC Sound (I2S) Driver. ESP-IDF Programming Guide. <https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/peripherals/i2s.html>. Accessed June 30, 2026
15. FreeRTOS: Deferred Interrupt Handling. FreeRTOS Kernel Documentation. <https://www.freertos.org/Documentation/02-Kernel/02-Kernel-features/11-Deferred-interrupt-handling>. Accessed June 30, 2026